

Diseño e implementación de una Unidad Aritmético Lógica (ALU) de 4 bits en SystemVerilog utilizando la tarjeta FPGA BASYS3

Ingeniería en Telecomunicaciones, Sistemas y Electrónica, Facultad de Estudios Superiores Cuautitlán, Universidad Nacional Autónoma de México

Abstract

En este diseño se diseñó y próximamente se implementará una Unidad Aritmético Lógica (ALU) de 4 bits utilizando SystemVerilog y el entorno de desarrollo Vivado. La ALU fue diseñada usando lógica combinacional, permitiendo realizar ocho operaciones: suma, resta, AND, OR, XOR, NOT, desplazamiento lógico a la derecha y desplazamiento lógico a la izquierda. Además, también se implementaron las banderas de acarreo, cero, desbordamiento y préstamo.

Se comprobó el correcto funcionamiento de las operaciones y de las banderas mediante una simulación, donde se hicieron diferentes combinaciones de entradas para verificar que la salida y las banderas funcionaran de manera correcta. Posteriormente, se generó el diseño RTL y se agregaron las entradas mediante los switches y las salidas mediante los LEDs correspondientes, para posteriormente conectar y probar el diseño en la tarjeta BASYS3.

Introducción

Una Unidad Aritmético Lógica (ALU) es una parte importante de los sistemas digitales, ya que permite realizar diferentes operaciones aritméticas y lógicas utilizando datos binarios. Para poder realizar cada una de estas operaciones se utiliza una señal de control, la cual indica qué operación debe realizar la ALU con las entradas.

En esta práctica se diseñó una ALU de 4 bits utilizando SystemVerilog y el entorno de desarrollo Vivado. El diseño se realizó utilizando lógica combinacional, por lo que no es necesario utilizar una señal de reloj ni un reset. La operación que se quiere realizar se selecciona mediante una entrada de 3 bits, lo que permite tener ocho operaciones diferentes.

Además del resultado de 4 bits, la ALU cuenta con cinco banderas: carry, borrow, overflow, zero y negative. Estas banderas permiten saber si se presentó alguna condición durante la operación, como un acarreo, un préstamo, un desbordamiento o si el resultado obtenido es cero o negativo.

Para comprobar que la ALU funcionara correctamente, se realizó una simulación en Vivado utilizando diferentes combinaciones de valores para las entradas. Con esto se pudo verificar el resultado de cada una de las operaciones y el comportamiento de las diferentes banderas. Posteriormente, se generó el diseño RTL y se agregaron las restricciones en el archivo .xdc, asignando los switches como entradas y los LEDs como salidas, dejando el diseño preparado para posteriormente conectarlo y probarlo en la tarjeta BASYS3.

Diseño general y funciones internas de la ALU

La ALU diseñada tiene como entradas dos valores de 4 bits, llamados A y B , y una entrada de 3 bits llamada *operation*, que se utiliza para seleccionar la operación que se va a realizar.

Como salidas se tiene un resultado de 4 bits llamado *out_put* y cinco banderas: carry, borrow, overflow, zero y negative.

El funcionamiento de la ALU se realizó mediante un bloque *always_comb*, ya que todas las operaciones utilizan lógica combinacional. Dentro de este bloque se utiliza una estructura *case* que permite seleccionar una de las ocho operaciones dependiendo del valor de la entrada *operation*.

Las operaciones que realiza la ALU son las siguientes:

- 000: realiza una suma entre A y B .
- 001: realiza una resta entre A y B .
- 010: realiza la operación AND entre A y B .
- 011: realiza la operación OR entre A y B .
- 100: realiza la operación XOR entre A y B .
- 101: realiza la operación NOT únicamente sobre A .
- 110: realiza un desplazamiento lógico de A un bit hacia la izquierda.
- 111: realiza un desplazamiento lógico de A un bit hacia la derecha.

Para la suma se utilizó una variable llamada *temp* de 5 bits. Esto permite guardar el quinto bit que se genera cuando la suma produce un acarreo. Los cuatro bits inferiores se utilizan como resultado de la operación y el quinto bit se utiliza para la bandera carry.

En la resta, la bandera borrow se activa cuando A es menor que B . También se utiliza la bandera overflow para detectar cuando el resultado de una operación aritmética no puede representarse correctamente con 4 bits en complemento a dos.

Las banderas zero y negative se calculan después de seleccionar la operación. La bandera zero se activa cuando el

resultado es igual a cero, mientras que `negative` toma el valor del bit más significativo del resultado, que corresponde al bit de signo cuando se trabaja con números en complemento a dos.

También se comprobó el comportamiento de las operaciones de desplazamiento. Para el desplazamiento lógico a la izquierda se utilizó el valor 0101, obteniendo como resultado 1010. Debido a que el bit más significativo del resultado es 1, la bandera `negative` se activa. Para el desplazamiento lógico a la derecha se utilizó el valor 1010, obteniendo como resultado 0101, por lo que en este caso la bandera `negative` permanece en cero.

Análisis de resultados de la simulación

En la simulación se realizaron diferentes pruebas para comprobar el funcionamiento de las ocho operaciones de la ALU y de las banderas. Primero se realizó una suma de $0011 + 0010$, obteniendo como resultado 0101, sin activar ninguna bandera. Después se probó una suma con acarreo utilizando $1111 + 0001$, donde el resultado fue 0000 y la bandera `carry` se activó debido al quinto bit generado en la suma. También se realizó una suma con desbordamiento utilizando $0111 + 0001$, obteniendo 1000 y activando la bandera `overflow`.

Para la resta se probó primero $0110 - 0010$, obteniendo 0100 sin activar las banderas de `borrow` ni `overflow`. Después se realizó una resta donde A es menor que B , utilizando $0011 - 0101$. El resultado obtenido fue 1110 y se activó la bandera `borrow`. También se probó una resta con desbordamiento utilizando $0111 - 1111$, obteniendo 1000 y activando la bandera `overflow`.

Posteriormente se probaron las operaciones lógicas. Para la operación AND se utilizaron 1100 y 1010, obteniendo 1000. En la operación OR se obtuvo 1110 y para XOR se obtuvo 0110. En el caso de NOT se utilizó 1010 como entrada de A , obteniendo 0101 como resultado.

Finalmente se comprobaron los desplazamientos. Para el desplazamiento lógico a la izquierda se utilizó 0101, obteniendo 1010 y activando la bandera `negative`, debido a que el bit más significativo del resultado es 1. Para el desplazamiento lógico a la derecha se utilizó 1010, obteniendo 0101, por lo que la bandera `negative` permaneció en cero.

Con estos resultados se pudo comprobar mediante la simulación que las operaciones seleccionadas por la entrada `operation` producen los resultados esperados y que las banderas cambian de acuerdo con las condiciones de cada operación. También se comprobó el funcionamiento de la bandera `zero`, la cual se activa cuando el resultado es 0000, como ocurrió en la suma $1111 + 0001$.

Tabla 1: Resultados obtenidos durante la simulación de la ALU

A	B	Operación	Resultado	Bandera activada
0011	0010	Suma	0101	Ninguna
1111	0001	Suma	0000	Carry, Zero
0111	0001	Suma	1000	Overflow, Negative
0110	0010	Resta	0100	Ninguna
0011	0101	Resta	1110	Borrow, Negative
0111	1111	Resta	1000	Overflow, Negative
1100	1010	AND	1000	Negative
1100	1010	OR	1110	Negative
1100	1010	XOR	0110	Ninguna
1010	0000	NOT	0101	Ninguna
0101	0000	Desplazamiento izquierda	1010	Negative
1010	0000	Desplazamiento derecha	0101	Ninguna



Figura 1: Simulación de las operaciones y banderas de la ALU .

Conclusiones

Con esta actividad pude entender mejor cómo funciona una ALU y cómo se pueden juntar diferentes operaciones dentro de un mismo diseño dependiendo de una señal de selección. Aunque las operaciones como suma, resta, AND, OR o XOR por separado son sencillas, al integrarlas en un solo sistema también fue necesario considerar qué información adicional se debía mostrar mediante las banderas.

La parte de las banderas también fue importante, ya que no solamente se obtiene un resultado de 4 bits, sino que también se puede conocer qué ocurrió durante la operación. Por ejemplo, con las pruebas realizadas se pudo observar la diferencia entre un `carry`, un `borrow` y un `overflow`, además de comprobar cómo `zero` y `negative` dependen directamente del resultado. Al probar diferentes combinaciones de entradas se pudieron encontrar casos en los que las banderas cambiaban como en las sumas y restas con desbordamiento, y también se pudo comprobar el comportamiento de los desplazamientos. Esto permitió tener mayor seguridad de que la lógica implementada estaba funcionando de la forma esperada.

Apéndice A. Código de la ALU

En este apartado se incluye el código en SystemVerilog utilizado para realizar el diseño de la ALU de 4 bits

```

module alu(
//Declaración de entradas y salidas

```

```

input logic [3:0] A,B,
input logic [2:0] operation,
output logic [3:0] out_put,
output logic carry,borrow,overflow,zero,negative
);
//Guardar 5to bit
logic [4:0] temp;
//Lógica de la ALU
always_comb begin

```

```

zero=(out_put==4'b0000);
//Negative
negative = out_put[3];
end
endmodule

```

Diagrama RTL

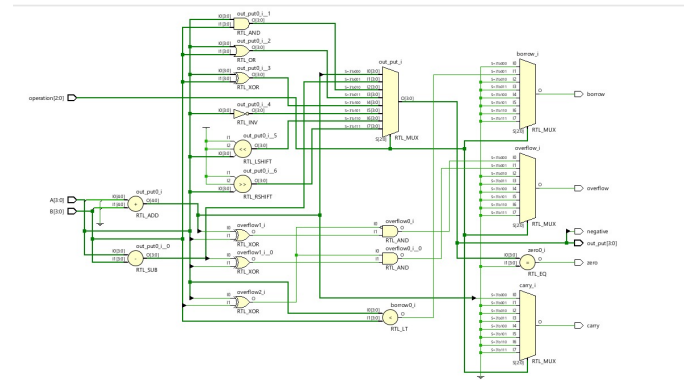


Figura A.2: Diagrama RTL de la ALU generado en Vivado.

```

//valores iniciales de las banderas
carry    =1'b0;
borrow   =1'b0;
overflow =1'b0;
zero     =1'b0;
negative =1'b0;

case (operation)
// Suma
3'b000: begin
temp = {1'b0, A} + {1'b0, B};
out_put = temp[3:0];
carry = temp[4];
overflow = (~(A[3]^B[3])) & (out_put[3]^A[3]);
end
// Resta
3'b001: begin
out_put=A-B;
borrow =(A<B);
overflow=(A[3]^B[3]) & (out_put[3]^A[3]);
end
//AND
3'b010:begin
out_put=A&B;
end
//OR
3'b011:begin
out_put=A|B;
end
//XOR
3'b100:begin
out_put=A^B;
end
// NOT de A
3'b101:begin
out_put=~A;
end
//Desplazamiento lógico a la izquierda
3'b110:begin
out_put=A<<1;
end
//Desplazamiento lógico a la derecha
3'b111:begin
out_put=A>>1;
end
endcase
//Zero

```